

Q.No:4

Problem Description

You are given a weighted graph with **N** vertices and **M** edges. Find the total weight of its **maximum** spanning tree.

Constraints

- $1 \leq T \leq 20$
- $1 \leq N \leq 5000$
- $1 \leq M \leq 100\,000$
- $1 \leq c \leq 10\,000$

Input

The first line contains one integer **T** denoting the number of test cases. Each test case starts with a line containing 2 space-separated integer: **N** and **M**. Each of the following **M** lines contain description of one edge: three different space-separated integers: **a**, **b** and **c**. **a** and **b** are different and from 1 to **N** each and denote numbers of vertices that are connected by this edge. **c** denotes weight of this edge.

Output

For each test case output one integer - the total weight of its **maximum** spanning tree.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
3 3
1 2 2
2 3 3
1 3 4
```

EXPECTED OUTPUT

```
7
```

Test Case 2

INPUT (STDIN)

```
1
3 3
1 2 3
2 3 4
1 3 5
```

EXPECTED OUTPUT

```
9
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int printheap(int N)
```

Complexity Test Cases

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 5005
#define MAXM 100005

typedef struct
{
    int u, v, w;
} Edge;

Edge edges[MAXM];
int parent[MAXN];
```

```
int heapSize;
int find(int x)
{
    if (parent[x] == x)
        return x;

    parent[x] = find(parent[x]);
    return parent[x];
}
int unite(int a, int b)
{
    a = find(a);
    b = find(b);

    if (a == b)
        return 0;

    parent[b] = a;
    return 1;
}

void swap(Edge *a, Edge *b)
{
    Edge temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < heapSize &&
        edges[left].w > edges[largest].w)
    {
        largest = left;
    }

    if (right < heapSize &&
        edges[right].w > edges[largest].w)
    {
        largest = right;
    }

    if (largest != i)
    {
        swap(&edges[i], &edges[largest]);
        heapify(largest);
    }
}

int printheap(int N)
{
    {
```

```
heapSize = N;

/* Build max heap */
for (int i = heapSize / 2 - 1; i >= 0; i--)
{
    heapify(i);
}

return heapSize;
}

/* Heap sort */
void heapSort(int N)
{
    printheap(N);

    for (int i = N - 1; i > 0; i--)
    {
        swap(&edges[0], &edges[i]);

        heapSize = i;
        heapify(0);
    }

}

int main()
{
    int T;

    scanf("%d", &T);

    while (T--)
    {
        int N, M;

        scanf("%d %d", &N, &M);
        for (int i = 0; i < M; i++)
        {
            scanf("%d %d %d",
                &edges[i].u,
                &edges[i].v,
                &edges[i].w);
        }

        for (int i = 1; i <= N; i++)
        {
            parent[i] = i;
        }

        heapSort(M);
    }
}
```

```
long long total = 0;
int count = 0;

printf("Ouptut:\n");
for (int i = M - 1; i >= 0; i--)
{
    if (unite(edges[i].u, edges[i].v))
    {
        total += edges[i].w;
        count++;
    }

    if (count == N - 1)
        break;
}

printf("%lld\n", total);

return 0;
}
```

Output:

Test_case:1

```
● aadityajaiswal@Aadityas-Ma
1
3 3
1 2 2
2 3 3
1 3 4
Ouptut:
7
○ aadityajaiswal@Aadityas-Ma
```

Test_case:2

```
● aadityajaiswal@Aad
1
3 3
1 2 3
2 3 4
1 3 5
Ouptut:
9
○ aadityajaiswal@Aad
```